MODELING EARTHQUAKE SIGNIFICANCE: CURVE FITTING AND REGRESSION ANALYSIS OF GLOBAL SEISMIC DATA

PIERRE VISCONTI

Abstract. This project evaluates different linear and non-linear regression methods for modeling the United States Geological Survey (USGS) significance score (sig) of earthquake events. The initial modeling objective was to assess how well the variation in the response variable sig was explained by the three primary predictors, magnitude, maximum instrumental intensity (MMI), and maximum reported felt intensity (CDI). After establishing this baseline, a more complex model was constructed by incorporating additional predictors to determine whether the explanatory power for sig could be improved. Based on the three linear methods, Cholesky Decomposition, QR Decomposition, and the LSQR solver in MATLAB, the more complex model had more explanatory power than the basic one (R^2 of 0.52 versus 0.46).

1. Introduction

Curve fitting is "the process of constructing a curve, or mathematical function, that has the best fit to a series of data points." The fitted curves can be used to assist in data visualization, "to infer values of a function where no data are available, and to summarize the relationships among two or more variables" [4]. Curve fitting is a fundamental tool in data analysis, enabling researchers to model relationships between variables, to quantify trends, and make predictions based on observed data. In seismology (a science that deals with earthquakes and with artificially produced vibrations of the earth [3]), earthquake events are characterized by multiple interacting factors such as magnitude, depth, and intensity. Curve fitting offers a way to evaluate how well composite indices, such as the significance score, reflect underlying drivers. The significance score is officially derived from the magnitude of the earthquake, the instrumental intensity (MMI), the felt intensity (CDI), and the estimated impact [1]. The experimental question guiding this project is whether the significance score can be more accurately predicted when additional variables are incorporated into regression models. By comparing baseline and extended curve fits, we seek to assess the performance of the current formulation and explore whether additional predictors provide meaningful improvements.

2. Methods and Computations

2.1. **Dataset.** The dataset analyzed for this project contains records of 782 earthquakes occurring worldwide between January 1, 2001, and January 1, 2023. The dataset was compiled from the United States Geological Survey (USGS) earthquake catalog via API requests, which provides standardized event information. Each observation includes title: title name given to the earthquake, magnitude: the magnitude of the earthquake, date_time: date and time, cdi: the maximum reported intensity for the event range, mmi: the maximum estimated instrumental intensity for the event, tsunami: "1" for events in oceanic regions and "0" otherwise, sig: a number describing how

significant the event is, nst: the total number of seismic stations used to determine earthquake location, dmin: horizontal distance from the epicenter to the nearest station, gap: the largest azimuthal gap between azimuthally adjacent stations (in degrees), magType: the method or algorithm used to calculate the preferred magnitude for the event, depth: the depth where the earthquake begins to rupture, latitude / longitude: coordinate system by means of which the position or location of any place on Earth's surface can be determined and described [1]. Other variables are also included in the data set, but they had a significant amount of missing values and were therefore excluded from this project. This data set is particularly well suited for regression and curve fitting because it combines continuous predictors (magnitude, depth, station coverage) with categorical descriptors (tsunami), allowing for a comprehensive evaluation of how different factors contribute to the reported significance of seismic events.

2.2. **Comparison of Linear Algorithms.** Three different linear least-squares methods were applied to the earthquake dataset. Specifically, Cholesky decomposition was used to solve the normal equations, QR decomposition was used to factor the design matrix into orthonormal and triangular components, and MATLAB's built-in lsqr solver as a benchmark. The initial modeling objective was to assess how well the variation in the response variable sig (a composite score given by the USGS to quantify the overall significance of an earthquake event), was explained by the three primary predictors, magnitude, maximum instrumental intensity (MMI), and maximum reported felt intensity (CDI). After establishing this baseline, a more complex model was constructed by incorporating additional predictors to determine whether the explanatory power for sig could be improved. The additional predictors consist of a binary variable indicating if a tsunami occurred, the "algorithm used to calculate the preferred magnitude for the event", the depth where the earthquake began to rupture, and the longitude and latitude coordinates of the event [1]. These were selected because it makes logical sense for them to have some impact on the significance of an earthquake. For example, the presence of a tsunami would likely make the event more significant, as would longitude and latitude coordinates corresponding to an area with weak infrastructure. The following section provides a mathematical background for each least-squares method.

Theorem 2.1. *If an $n \times n$ matrix A is SPD, there exists an upper triangular matrix R (Cholesky Factor) such that*

$$A = R^T R$$

which is called the Cholesky Decomposition of A [2].

The Cholesky Decomposition for Symmetric Positive Definite (SPD) matrices provides a backwards stable algorithm. This method is similar to LU Decomposition but unlike LU the matrix R^T, as described in Theorem 2.1, does not contain ones on its diagonal. The normal equations, in the context of least squares, are given as

$$A^T A\vec{x} = A^T\vec{b}.$$

If $A^T A$ is SPD, then Theorem 2.1 applies and we apply the Cholesky Decomposition where

$$A^T A = R^T R.$$

To solve the system

$$R^T R\vec{x} = \vec{b}$$

we let $\vec{y} = R\vec{x}$ and solve $R^T\vec{y} = \vec{b}$ by forward substitution, and then solve $R\vec{x} = \vec{y}$ by backward substitution. For this method we want A to be well-conditioned and for rank(A)=n [2].

QR Decomposition is better suited when rank(A)=n, but A is ill-conditioned. Rather than forming $A^T A$, we factor A where Q is an orthonormal matrix of size $m \times m$ and R is an $n \times n$ upper triangular matrix that satisfies the equation

$$A = Q \begin{pmatrix} R \\ 0 \end{pmatrix}.$$

The system then becomes

$$R\vec{x} = Q^T \vec{b}$$

which is solved using backwards substitution [2].

Lastly is the lsqr() function built into MATLAB. "The least squares (LSQR) algorithm is an adaptation of the conjugate gradients (CG) method for rectangular matrices." It is "the only iterative linear system solver that can handle rectangular and inconsistent coefficient matrices." By default $x = \text{lsqr}(A, b)$ will attempt to solve the system of linear equations $A * x = b$ for x using the Least Squares Method, where it finds a least squares solution for x that minimizes norm$(b - A * x)$. "When A is consistent, the least squares solution is also a solution of the linear system. When the attempt is successful, lsqr displays a message to confirm convergence. If lsqr fails to converge after the maximum number of iterations or halts for any reason, it displays a diagnostic message that includes the relative residual norm$(b - A * x)$/norm(b) and the iteration number at which the method stopped" [6].

The Cholesky and QR methods were coded and applied to the earthquake dataset using MATLAB. Modeling the response variable sig (significant scores) with the three primary predictors was performed first as a baseline model to evaluate their performance in explaining the variance in the response. A plot of the observed significance scores versus the predicted significance score was generated for each method (given as dots with a 1:1 line plotted on top) along with the computed R^2 value to use as a metric of performance. The plotted dots should ideally match the 1:1 line as close as possible. This can be seen in Figure 1. Using the lsqr function, another plot and R^2 value was generated as a benchmark to compare the other methods against, this can be seen in Figure 2 (A).

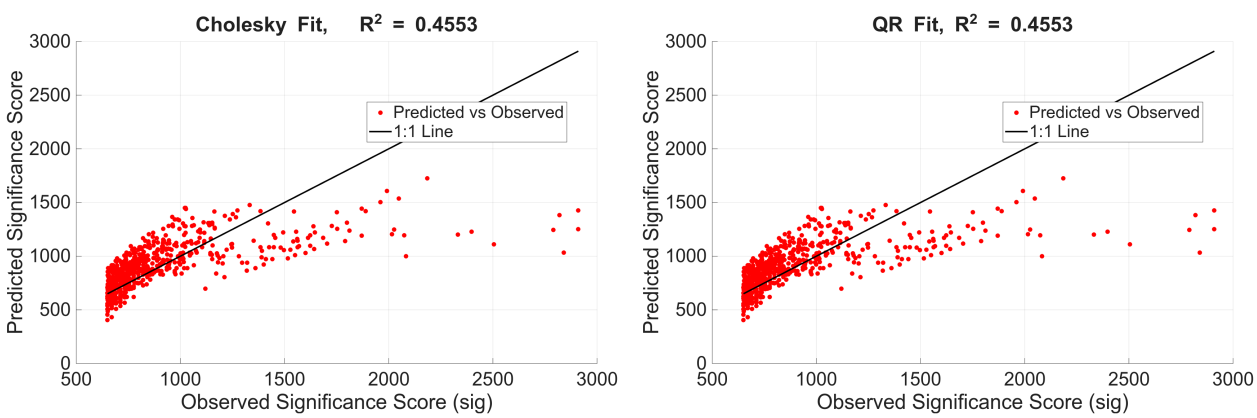
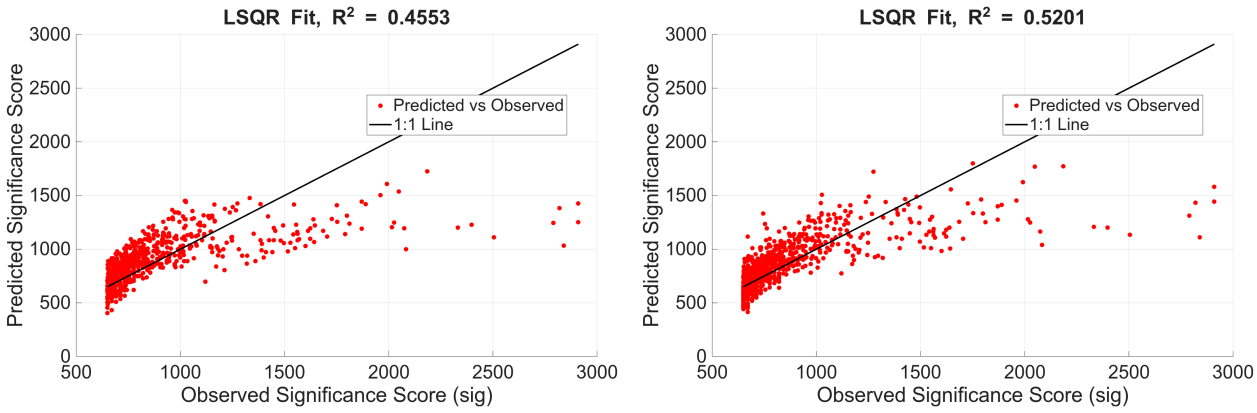


FIGURE 1. Performance of the Cholesky and QR fits on the basic sig model.

After establishing this baseline, a more complex model was constructed by incorporating additional predictors to determine whether the explanatory power for sig could be improved. The additional predictors consist of a binary variable indicating if a tsunami occurred, the "algorithm



(A) Basic sig model. (B) Updated sig model.

FIGURE 2. Benchmark LSQR Performance.

used to calculate the preferred magnitude for the event", the depth where the earthquake began to rupture, and the longitude and latitude coordinates of the event. Once again, all three methods were coded and applied to the dataset using MATLAB and then plotted in the same manner as the basic model. The plots can be seen in Figure 3 with the baseline lsqr provided in Figure 2 (B).

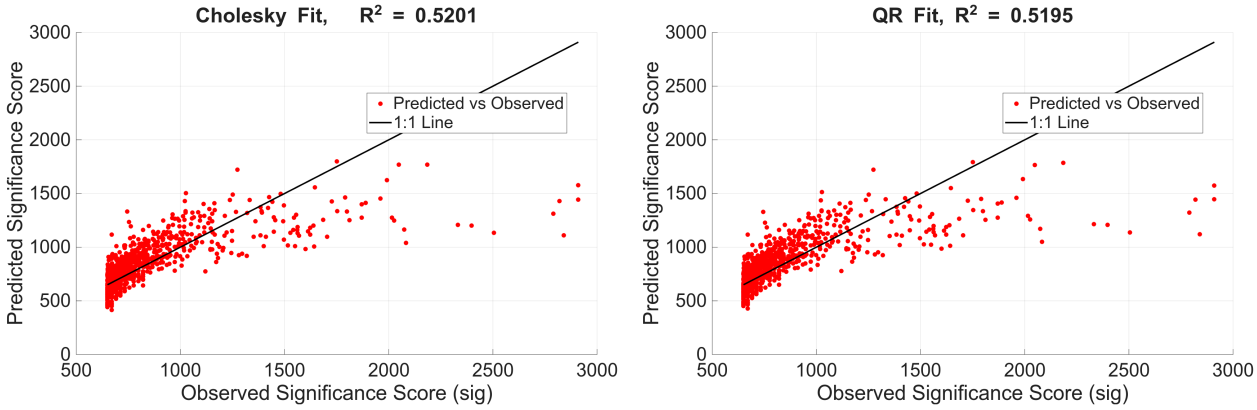


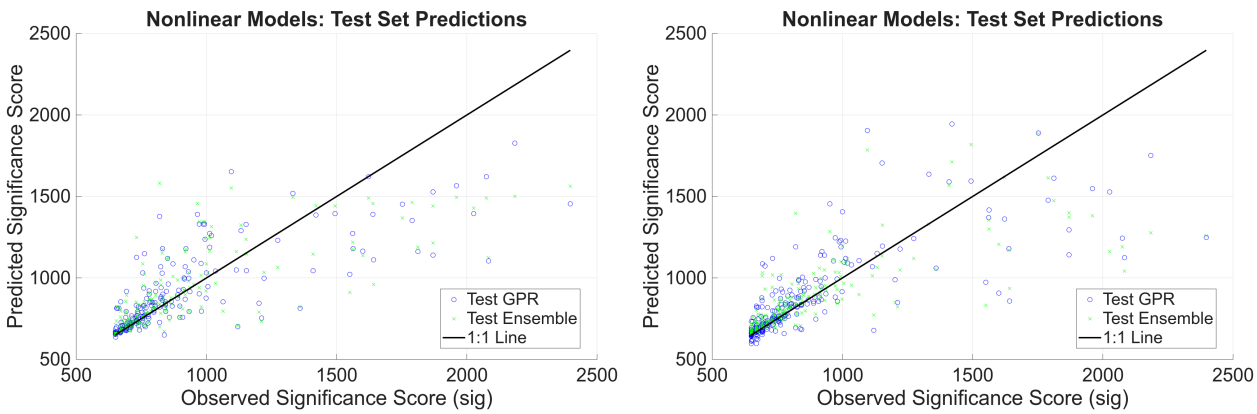
FIGURE 3. Performance of the Cholesky and QR Fits on the Updated Sig Model.

2.3. **Condition Number.** Using the cond() function in MATLAB, the condition number of the matrix A was found to be $1.65e+02$ for the basic model and $1.34e+18$ for the updated model. The max absolute difference and R^2 values can be seen in the Summary of Results section in Table 2. The basic model has a relatively low condition number which suggests that matrix A is well-conditioned and should be suitable for use with Cholesky, QR, and LSQR methods. This is backed up by the results in Table 2 for the max absolute errors between methods for the basic model. The values remain extremely small between methods and the R^2 values are indentical, which demonstrates how Cholesky and QR methods yield reliable results when using a suitable matrix.

For the updated model, the condition of the matrix A was very large, indicating extreme ill-conditioning. This instability was clearly reflected in the updated model results the maximum

absolute differences between Cholesky and QR versus LSQR reached very large numbers. This highlights the importance of conditioning in regression. When the matrix is well-conditioned, unstable methods like Cholesky yield reliable results, but in high-dimensional matrices, LSQR is far more robust making it the better choice for ill-conditioned matrices.

2.4. **Other Curve Fits.** A lot of other curve fits available, beyond linear, are only suitable for tasks where A is a low-dimensional. For this project, due to a larger number of predictors, more advanced algorithms were better suited for the task. We selected Gaussian Process Selection (GPR) and Ensemble Regression Trees. While these are not traditional curve fitting algorithms, they still have a curve-fitting role and the output is a fitted curve. "Gaussian Process Regression (GPR) is a powerful and flexible non-parametric regression technique used in machine learning and statistics. It is particularly useful when dealing with problems involving continuous data," [5] as is the case for this project. "A regression tree ensemble is a predictive model composed of a weighted combination of multiple regression trees. In general, combining multiple regression trees increases predictive performance" [7]. For these algorithms, the dataset was split into training and testing sets at a 70:30 split to check for overfitting. Plots of the observed significance score versus the predicted significance score are given in Figure 4, with the respective R^2 values for both the training and testing sets provided in Table 1.



(A) Basic sig model.

(B) Updated sig model.

FIGURE 4. Performance of the Cholesky and QR Fits on the Updated Sig Model.

Model (Basic)	Train R^2	Test R^2
GPR	0.5555	0.6277
Ensemble Trees	0.6190	0.5902
Model (Updated)	Train R^2	Test R^2
GPR	0.7788	0.5767
Ensemble Trees	0.7921	0.5927

TABLE 1. Nonlinear Model Performance on Basic and Expanded Feature Sets

3. Summary of Results

This project investigated the performance and stability of three linear least-squares algorithms (Cholesky decomposition, QR decomposition, and MATLAB's built-in lsqr solver) applied to a global earthquake dataset. The initial modeling objective was to assess how well three seismic predictors, (magnitude, maximum instrumental intensity (MMI), and maximum reported felt intensity (CDI), explain variation in the USGS-assigned significance score (sig). After establishing baseline performance, a more complex model incorporating additional predictors was constructed to improve explanatory power.

Method	R^2 **(Basic Model)**	R^2 **(Updated Model)**
Cholesky	0.455316	0.520139
QR	0.455316	0.519467
LSQR	0.455316	0.520129

Coefficient Pair	**Max Abs Diff (Basic)**	**Max Abs Diff (Updated)**
Cholesky vs QR	9.549694×10^{-11}	3.040458×10^{16}
Cholesky vs LSQR	7.485407×10^{-7}	1.373069×10^{3}
QR vs LSQR	7.485568×10^{-7}	3.040458×10^{16}

Table 2. Comparison of Least-Squares Methods: Basic vs. Updated Model

Model (Basic)	Train R^2	Test R^2
Cholesky	0.4382	0.4924
QR	0.4382	0.4924
LSQR	0.4382	0.4924
Model (Updated)	Train R^2	Test R^2
Cholesky	0.5115	0.5323
QR	0.5115	0.5327
LSQR	0.5114	0.5322

Table 3. Linear Model Performance with Training and Testing Sets.

For the linear least-squares methods, the basic model using only magnitude, MMI, and CDI produced identical results across Cholesky, QR, and LSQR. As shown in Table 2, all three methods achieved an R^2 value of about 0.455. In Table 3, where the dataset was split into training and testing sets to check for overfitting, the R^2 values are again all the same between methods, even on the testing set. These results, in combination with matrix A having a small condition number, demonstrates how these methods perform well when A is well-conditioned, as confirmed by the small maximum absolute difference values in Table 2.

When the updated model was introduced, incorporating a tsunami binary variable, magnitude algorithm used, earthquake rupture depth, and geographic coordinates, the condition number increased dramatically. This extreme ill-conditioning was reflected in the coefficient comparisons in Table 2, where the maximum absolute difference between Cholesky and QR reached 3.04×10^{16}, and 1.37×10^{3} between Cholesky and LSQR. Despite these instabilities, the R^2 values across methods

remained similar, as shown in Table 2 and in Table 3 for the training/testing results. Importantly, the updated model performed better for all three linear methods than the basic sig model, as can be seen by the higher R^2 values for both the training and testing sets. The testing set results were actually higher than the training set which indicates little to no overfitting. Despite these results, we recommend avoiding the Cholesky and QR decompositions for the updated model due to the ill-conditioned matrix A and should instead stick with LSQR for more reliable results.

For the nonlinear models, Gaussian Process Regression (GPR) and Ensemble Regression Trees were evaluated. These curve fitting algorithms performed slightly better on the basic sig model compared to the updated one as seen in Table 2. However, in Table 3, both methods achieved higher training performance, 0.7788 for GPR and 0.7921 for Ensemble, but test performance declined to 0.5767 and 0.5927, respectively. This pattern indicates that the nonlinear models captured more variance in the training data but at the cost of overfitting, particularly for GPR. Although further tuning of the model parameters could alleviate this problem as the default parameters were used for this project.

4. Conclusions

The results demonstrate that conditioning strongly dictates the reliability of linear solvers, with LSQR proving most robust under ill-conditioning, while nonlinear models can improve training fit but risk overfitting when additional predictors are introduced. For well-conditioned problems, methods such as Cholesky Decomposition and QR Decomposition are effective and accurate, but as the number of predictors grow, the probability of having columns that are not linearly independent increases and more robust methods, such as LSQR, become essential.

A lesson to watch out for, especially for data or graphs displayed on the news, is to be aware of overfitting. Caution should be used whenever using data that was not split into training and testing sets, as there could be extreme overfitting occurring without being known. This means that the model might be highly effective on the dataset itself, but useless on similar data outside of the dataset it was trained on. A curve that hugs the data perfectly may be overfit, and reported metrics can be misleading if the underlying model is unstable. Understanding concepts like condition number and overfitting helps us critically evaluate whether a reported trend is robust or just a product of the chosen method.

References

[1] Chirag Chauhan. Earthquake dataset. `https://www.kaggle.com/datasets/warcoder/earthquake-dataset/data`, 2023. Accessed: 2025-11-02.

[2] Breschine Cummins. Numerical linear algebra and optimization. Lecture notes, M441, Montana State University, 2025.

[3] Merriam-Webster. Seismology. `https://www.merriam-webster.com/dictionary/seismology`, 2025. Accessed: 2025-11-02.

[4] Kumar Molugaram and G. Shanker Rao. Chapter 5 - curve fitting. In Kumar Molugaram and G. Shanker Rao, editors, *Statistical Techniques for Transportation Engineering*, pages 281–292. Butterworth-Heinemann, 2017.

[5] Isha Phadatare. Gaussian process regression (gpr), August 2025. Accessed: 2025-11-02.

[6] The MathWorks, Inc. *lsqr: Solve linear system using LSQR algorithm*. MathWorks, 2025. Accessed: 2025-11-02.

[7] The MathWorks, Inc. *Regression Tree Ensembles - MATLAB & Simulink*. MathWorks, 2025. Accessed: 2025-11-02.

```
function least_squares()
    % Import the dataset as a table
    eqData = import_data("earthquake_data.csv");

    % Convert magType to a dummy variable
    magType = categorical(eqData.magType);
    magType = dummyvar(magType);

    % Initialize the matrix A and vector b
    %X = [eqData.magnitude, eqData.mmi, eqData.cdi]; % Original formulation
    X = [eqData.magnitude, eqData.mmi, eqData.cdi, eqData.tsunami, magType,
eqData.depth, eqData.latitude, eqData.longitude];
    A = [ones(size(X,1),1), X];
    b = eqData.sig;

    %%%%%%%%%%%%%%%%%%%%%%%%%
    % CHOLESKY DECOMPOSITION
    %%%%%%%%%%%%%%%%%%%%%%%%%
    beta_chol = cholesky(A, b);
    yhat = A * beta_chol;
    residuals = b - yhat;

    % Compute R^2
    R2_chol = 1 - sum(residuals.^2)/sum((b - mean(b)).^2);

    % Plot the observed values versus the predicted
    figure(1); clf;
    scatter(b, yhat, 'ro', 'filled'); hold on;
    plot([min(b), max(b)], [min(b), max(b)], 'k-', 'LineWidth', 2); % 1:1 line
    xlabel('Observed Significance Score (sig)');
    ylabel('Predicted Significance Score');
    title(sprintf('Cholesky Fit,   R^2 = %.4f', R2_chol));
    legend('Predicted vs Observed','1:1 Line','Location','best');
    grid on;
    drawnow;

    %%%%%%%%%%%%%%%%%%%
    % QR DECOMPOSITION
    %%%%%%%%%%%%%%%%%%%
    beta_qr = qr_decomp(A, b);
    yhat_qr = A * beta_qr;
    residuals = b - yhat_qr;

    % Compute R^2
    R2_qr = 1 - sum(residuals.^2)/sum((b - mean(b)).^2);

    % Plot the observed values versus the predicted
    figure(2); clf;
    scatter(b, yhat_qr, 'ro', 'filled'); hold on;
    plot([min(b), max(b)], [min(b), max(b)], 'k-', 'LineWidth', 2);
    xlabel('Observed Significance Score (sig)');
    ylabel('Predicted Significance Score');
```

```
title(sprintf('QR Fit, R^2 = %.4f', R2_qr));
legend('Predicted vs Observed','1:1 Line','Location','best');
grid on;

%%%%%%%%%%%%%%%%%%%%%%%%
% MATLAB BUILT-IN LSQR
%%%%%%%%%%%%%%%%%%%%%%%%
beta_lsqr = lsqr(A, b, 1e-6, 100);
yhat_lsqr = A * beta_lsqr;
residuals = b - yhat_lsqr;

% Compute R^2
R2_lsqr = 1 - sum(residuals.^2)/sum((b - mean(b)).^2);

% Plot the observed values versus the predicted
figure(3); clf;
scatter(b, yhat_lsqr, 'ro', 'filled'); hold on;
plot([min(b), max(b)], [min(b), max(b)], 'k-', 'LineWidth', 2);
xlabel('Observed Significance Score (sig)');
ylabel('Predicted Significance Score');
title(sprintf('LSQR Fit, R^2 = %.4f', R2_lsqr));
legend('Predicted vs Observed','1:1 Line','Location','best');
grid on;

%%%%%%%%%%%%%%%%%%%%%%%%
% PRINT COMPARISON DATA
%%%%%%%%%%%%%%%%%%%%%%%%
fprintf('\nR^2 Comparison:\n');
fprintf('Cholesky: %.6f\n', R2_chol);
fprintf('QR:       %.6f\n', R2_qr);
fprintf('LSQR:     %.6f\n', R2_lsqr);

fprintf('\nMax absolute difference in coefficients:\n');
fprintf('Cholesky vs QR:   %.6e\n', max(abs(beta_chol - beta_qr)));
fprintf('Cholesky vs LSQR: %.6e\n', max(abs(beta_chol - beta_lsqr)));
fprintf('QR vs LSQR:       %.6e\n', max(abs(beta_qr - beta_lsqr)));


%%%%%%%%%%%%%%%%%%%%%%%%%
% TRAIN/TEST DATASETS
%%%%%%%%%%%%%%%%%%%%%%%%%
% Parameters
rng(1);
n = size(A,1);              % number of observations
idx = randperm(n);          % random permutation of indices
train_frac = 0.7;           % 70% training
nTrain = round(train_frac * n);
train_idx = idx(1:nTrain);
test_idx  = idx(nTrain+1:end);

% Partition data
A_train = A(train_idx,:);
b_train = b(train_idx);
A_test  = A(test_idx,:);
```

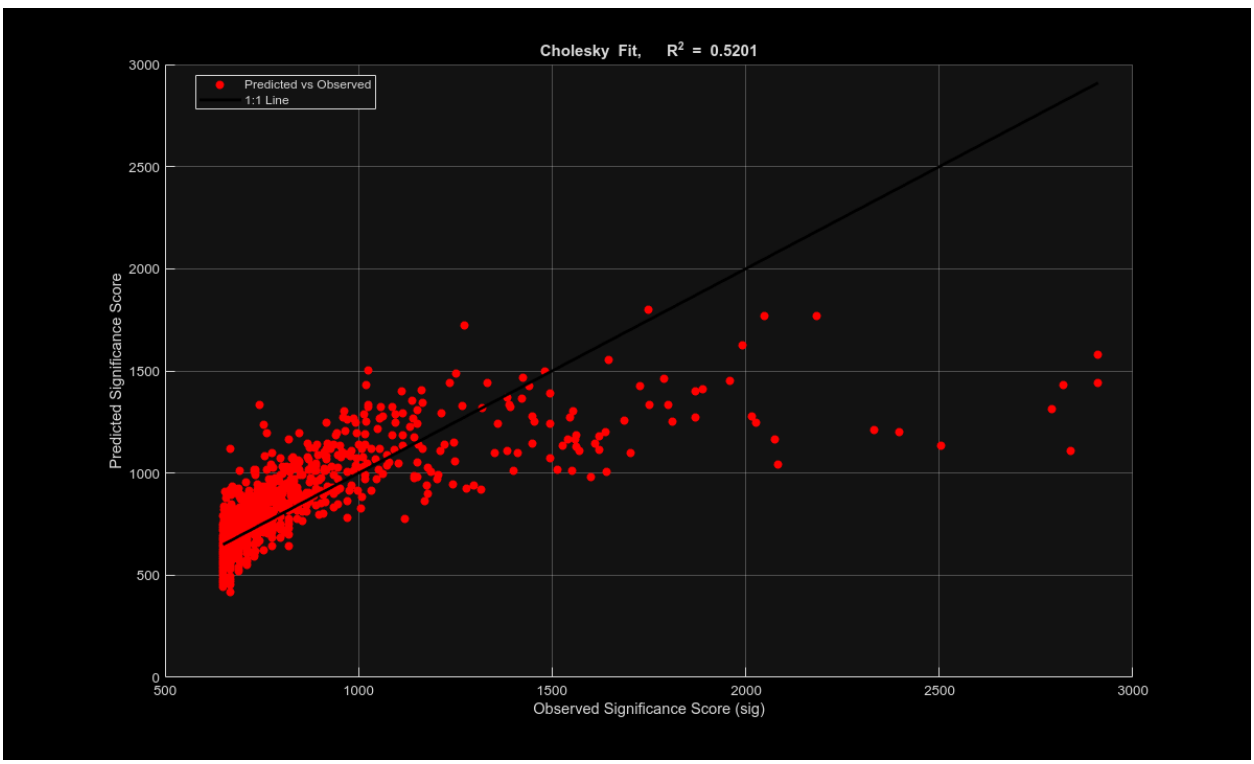
```
    b_test  = b(test_idx);

    % Cholesky
    beta_chol = cholesky(A_train, b_train);
    yhat_train_chol = A_train * beta_chol;
    yhat_test_chol  = A_test  * beta_chol;
    R2_train_chol = 1 - sum((b_train - yhat_train_chol).^2) / sum((b_train -
mean(b_train)).^2);
    R2_test_chol  = 1 - sum((b_test  - yhat_test_chol).^2)  / sum((b_test  -
mean(b_test)).^2);

    % QR
    beta_qr = qr_decomp(A_train, b_train);
    yhat_train_qr = A_train * beta_qr;
    yhat_test_qr  = A_test  * beta_qr;
    R2_train_qr = 1 - sum((b_train - yhat_train_qr).^2) / sum((b_train -
mean(b_train)).^2);
    R2_test_qr  = 1 - sum((b_test  - yhat_test_qr).^2)  / sum((b_test  -
mean(b_test)).^2);

    % LSQR
    beta_lsqr = lsqr(A_train, b_train, 1e-6, 100);
    yhat_train_lsqr = A_train * beta_lsqr;
    yhat_test_lsqr  = A_test  * beta_lsqr;
    R2_train_lsqr = 1 - sum((b_train - yhat_train_lsqr).^2) / sum((b_train -
mean(b_train)).^2);
    R2_test_lsqr  = 1 - sum((b_test  - yhat_test_lsqr).^2)  / sum((b_test  -
mean(b_test)).^2);

    % Print results
    fprintf('\nTraining vs Test R^2:\n');
    fprintf('Cholesky: Train = %.4f, Test = %.4f\n', R2_train_chol,
R2_test_chol);
    fprintf('QR:       Train = %.4f, Test = %.4f\n', R2_train_qr, R2_test_qr);
    fprintf('LSQR:     Train = %.4f, Test = %.4f\n', R2_train_lsqr,
R2_test_lsqr);


    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % COMPUTE THE CONDITION OF A
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    cond_A = cond(A);
    fprintf('\nCondition number of A: %.2e\n', cond_A);


    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Gaussian Process Regression (GPR)
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    gprMdl = fitrgp(A_train, b_train, ...
        'KernelFunction','squaredexponential', ...
        'Standardize',true);

    yhat_train_gpr = predict(gprMdl, A_train);
    yhat_test_gpr  = predict(gprMdl, A_test);
```

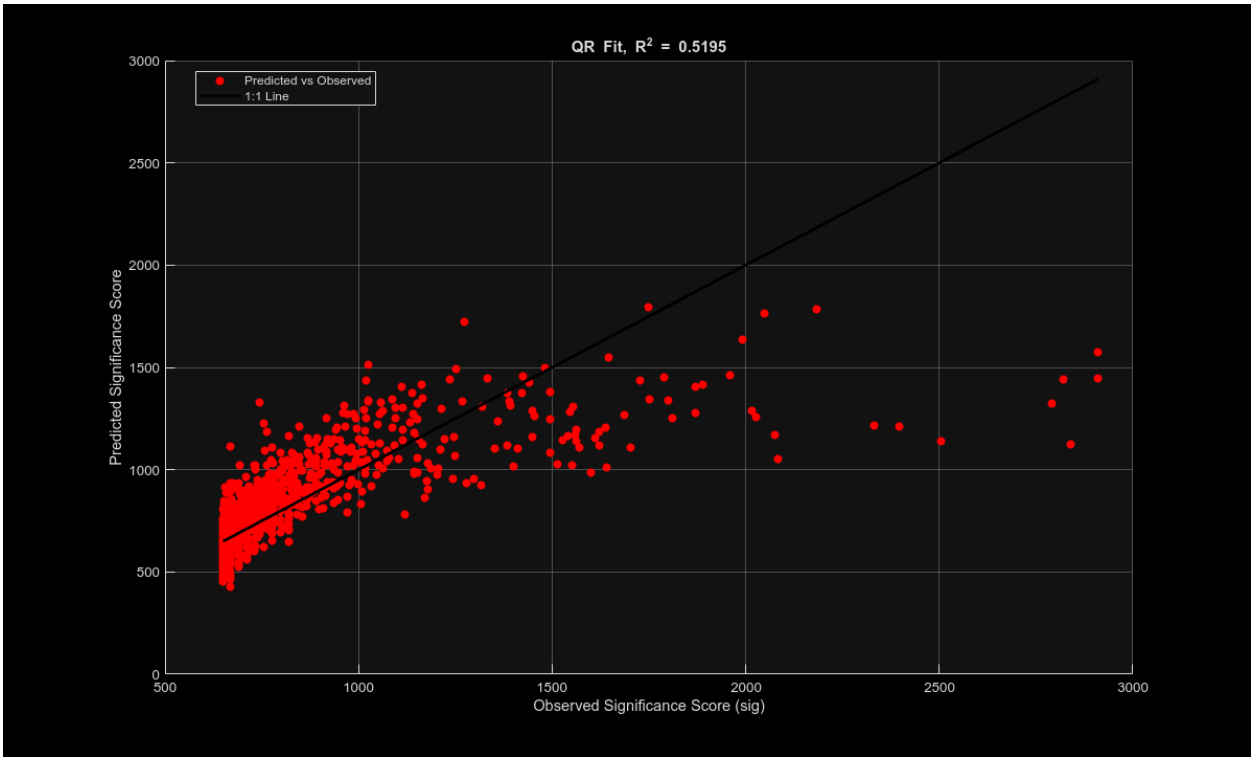
```
    R2_train_gpr = 1 - sum((b_train - yhat_train_gpr).^2) / sum((b_train -
mean(b_train)).^2);
    R2_test_gpr  = 1 - sum((b_test  - yhat_test_gpr).^2)  / sum((b_test  -
mean(b_test)).^2);

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Ensemble Regression Trees
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%
    ensMdl = fitrensemble(A_train, b_train, 'Method','Bag',
'NumLearningCycles',100);
    yhat_train_ens = predict(ensMdl, A_train);
    yhat_test_ens  = predict(ensMdl, A_test);
    R2_train_ens = 1 - sum((b_train - yhat_train_ens).^2) / sum((b_train -
mean(b_train)).^2);
    R2_test_ens  = 1 - sum((b_test  - yhat_test_ens).^2)  / sum((b_test  -
mean(b_test)).^2);

    % Report results
    fprintf('\nNonlinear Models: Training vs Test R^2\n');
    fprintf('GPR:       Train = %.4f, Test = %.4f\n', R2_train_gpr,
R2_test_gpr);
    fprintf('Ensemble:  Train = %.4f, Test = %.4f\n', R2_train_ens,
R2_test_ens);

    % Plot observed vs predicted for test set
    figure; hold on;
    scatter(b_test, yhat_test_gpr, 'bo', 'DisplayName','Test GPR');
    scatter(b_test, yhat_test_ens, 'gx', 'DisplayName','Test Ensemble');
    plot([min(b_test), max(b_test)], [min(b_test), max(b_test)], 'k-',
'LineWidth',2, 'DisplayName','1:1 Line');
    xlabel('Observed Significance Score (sig)');
    ylabel('Predicted Significance Score');
    title('Nonlinear Models: Test Set Predictions');
    legend('Location','best');
    grid on;
end


% Imports the dataset as a table and outputs the properties, variable
% names, and height and width.
function data = import_data(fileName)
    data = readtable(fileName);
    % View dataset attributes
    data.Properties
    data.Properties.VariableNames
    % Table specific functions
    height(data) % num rows
    width(data) % num cols
end


% Manually forms the normal equations, then uses chol() to do a Cholesky
% decomposition to solve the normal equations for the unknown intercept and
```

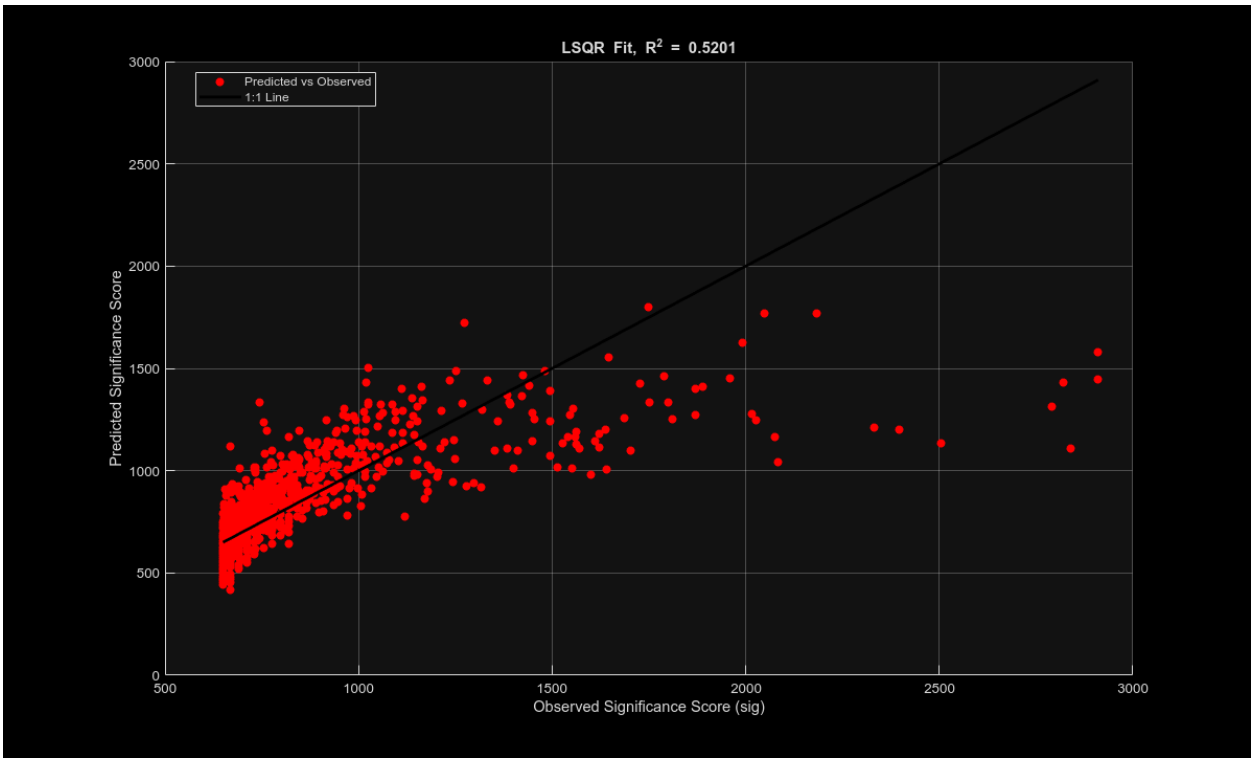
```
% slope of the best-fit line.
function x = cholesky(A,b)
    ATA = A' * A;
    ATb = A' * b;
    R = chol(ATA);
    z = fwdsub(R', ATb);
    beta = backsub(R, z);
    x = beta;
end

% Uses the QR decomposition of A to solve the least squares problem.
% factors A into Q (orthonormal columns) and R (upper triangular),
% then reduces the system to R*x = Q'*b, which is solved by back substitution.
function x = qr_decomp(A,b)
    [Q,R] = qr(A,0);
    rhs = Q' * b;
    x = backsub(R, rhs);
end


% This function solves the linear system Ax = b using forward substitution,
% where A has the properties that it is real, square, and lower triangular.
function x = fwdsub(A,b)
    if all(all(A == tril(A)))
        n = size(A,1);
        x(1) = b(1)/A(1,1);
        for i = 2:n
            bigsum = A(i,1:i-1)*x(1:i-1)';
            x(i) = (b(i) - bigsum) / A(i,i);
        end
        x = x';
    else
        n = size(A,1);
        x = NaN(n,1);
    end
end

% This function solves the linear system Ax = b using backward substitution,
% where A has the properties that it is real, square, and upper triangular.
function x = backsub(A,b)
    if all(all(A == triu(A)))
        n = size(A,1);
        x(n) = b(n)/A(n,n);
        for i = (n-1):-1:1
            bigsum = A(i,i+1:n)*x(i+1:n)';
            x(i) = (b(i) - bigsum) / A(i,i);
        end
        x = x';
    else
        n = size(A,1);
        x = NaN(n,1);
    end
end
```

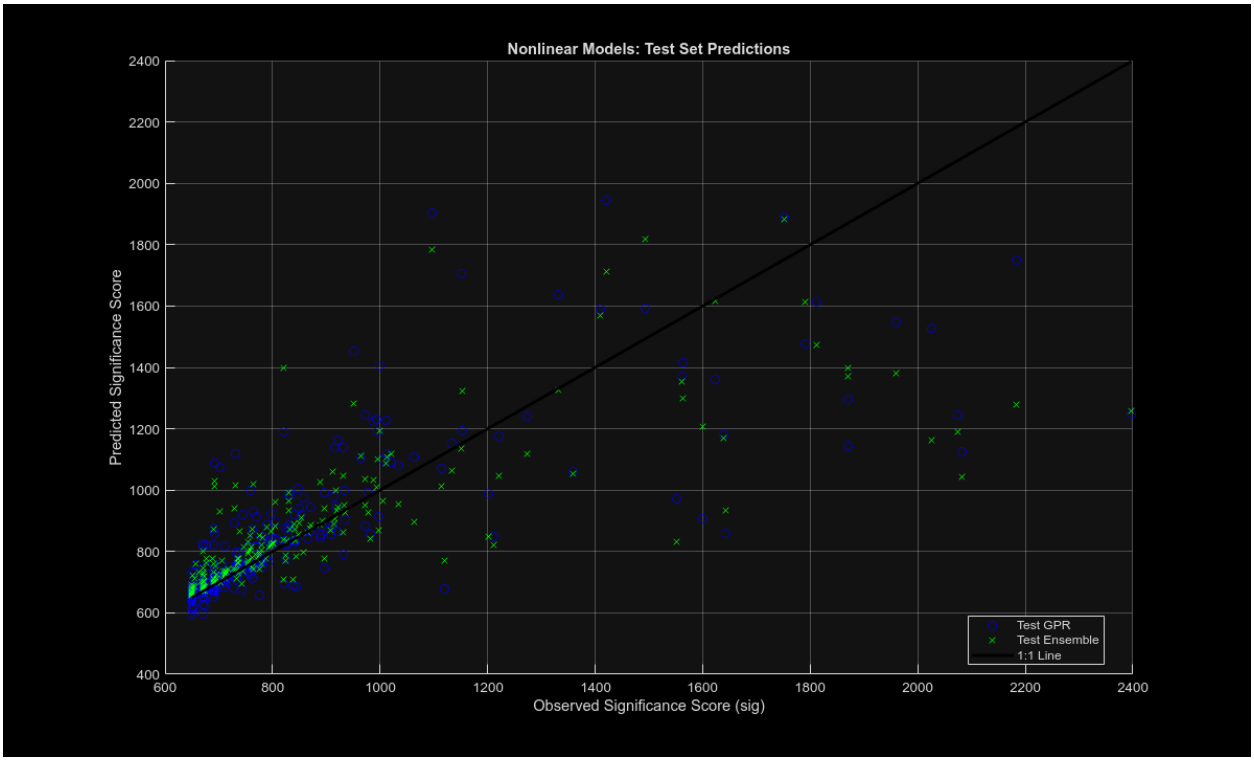
```
ans =

  TableProperties with properties:

             Description: ''
                UserData: []
          DimensionNames: {'Row'  'Variables'}
           VariableNames: {1×19 cell}
           VariableTypes: ["cell"    "double"    "cell"    …    ] (1×19
string)
    VariableDescriptions: {}
           VariableUnits: {}
      VariableContinuity: []
                RowNames: {}
        CustomProperties: No custom properties are set.
      Use addprop and rmprop to modify CustomProperties.


ans =

  1×19 cell array

  Columns 1 through 5

    {'title'}    {'magnitude'}    {'date_time'}    {'cdi'}    {'mmi'}

  Columns 6 through 11

    {'alert'}    {'tsunami'}    {'sig'}    {'net'}    {'nst'}    {'dmin'}

  Columns 12 through 16

    {'gap'}    {'magType'}    {'depth'}    {'latitude'}    {'longitude'}

  Columns 17 through 19

    {'location'}    {'continent'}    {'country'}


ans =

   782


ans =

    19

lsqr converged at iteration 26 to a solution with relative residual 0.24.

R^2 Comparison:
Cholesky: 0.520139
QR:       0.519467
```

```
LSQR:      0.520129

Max absolute difference in coefficients:
Cholesky vs QR:   3.040458e+16
Cholesky vs LSQR: 1.373069e+03
QR vs LSQR:       3.040458e+16
lsqr converged at iteration 26 to a solution with relative residual 0.24.

Training vs Test R^2:
Cholesky: Train = 0.5115, Test = 0.5323
QR:       Train = 0.5115, Test = 0.5327
LSQR:     Train = 0.5114, Test = 0.5322

Condition number of A: 1.34e+18

Nonlinear Models: Training vs Test R^2
GPR:       Train = 0.7788, Test = 0.5767
Ensemble:  Train = 0.7921, Test = 0.5927
```




QR Fit, R^2 = 0.5195
Predicted vs Observed
1:1 Line
Predicted Significance Score
Observed Significance Score (sig)
0
500
1000
1500
2000
2500
3000


LSQR Fit, R^2 = 0.5201
Predicted vs Observed
1:1 Line
Predicted Significance Score
Observed Significance Score (sig)
0
500
1000
1500
2000
2500
3000



Published with MATLAB® R2025a